Predicting Coronary Heart Disease from Survey Answers with Gradient Boosting in NumPy

Samuel Bachmann, Luca Civilla, Andrew Siminszky
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict coronary heart disease (MICHD) from the answers of 328,135 people to the BRFSS 2015 survey, scored by the F1 of the positive class (8.8% of the people). We decode the answer codes of the survey, compare the six methods of the course on the same features, and implement gradient boosted trees in NumPy. Every model predicts "positive" above a threshold chosen to maximize F1, not above 0.5. Our submitted model, boosting on 260 decoded columns, reaches an F1 of 0.444 ± 0.004 in 5-fold cross-validation and 0.454 on AIcrowd, against 0.438 and 0.447 for the regularized logistic regression. A heavier blend of boosted trees and neural networks gives our team's best AIcrowd score, 0.456. One question, asked only after a heart attack, gives away the label of 0.24% of the people; we use it and measure its effect.**

I. Introduction

The 321 columns of the data are answers, variables computed by the CDC from the answers, and fields of the survey administration [1]. Since only one person in eleven is positive, predicting "no" for everyone gives an accuracy of 0.912 but finds nobody, which is why the challenge scores the F1 of the positive class.

II. Data, Features and Validation

Decoding: The values are survey codes, not numbers: in `PHYSHLTH` ("days of poor physical health") 88 means none, 77 "don't know" and 99 "refused". We map "don't know", "refused" and "not asked" codes to missing values and "none" codes to 0, convert answers coded with a unit ("203" means 3 times per week), and re-number answers whose codes have no meaningful order (high blood pressure: 1 yes, 3 no, 4 borderline become 2, 0, 1). Many questions are asked only after a given answer (skip patterns), so whether a value is missing carries information.

Two inputs: The course methods use 96 questions selected by hand (43 secondary ones, e.g. on diet, vaccines or diabetes care), decoded by rules written for them: 11 nominal answers are one-hot encoded with "missing" as a level (levels with fewer than 200 rows dropped, the most frequent one as reference), numeric answers are median-imputed with a missing indicator when at least 10% are missing, and all features are standardized and clipped to ± 8 ($\approx$220 features). The gradient boosting reads 260 columns instead: all but 61 administrative (dates, sampling weights, phone screening) and redundant ones, decoded with tables written from the codebook of the provided data [1]; missing values get their own bin. All statistics are estimated on the training rows of each fold.

Model	Setting	CV F1	AIcrowd
Part 1: course methods, same ≈220 features			
least squares	–	0.4237 ± 0.0029	
ridge	$\lambda = 10^{-5}$	0.4237 ± 0.0029	
GD on MSE	100 iter., $\gamma = 0.9/L$	0.4223 ± 0.0029	
SGD on MSE	$3N$ steps, $\gamma = 2 \cdot 10^{-5}$	0.4211 ± 0.0031	
logistic	500 iter., $\gamma = 1$	0.4384 ± 0.0038	
reg. logistic	same, $\lambda_C = 10$	0.4384 ± 0.0038	0.447
Part 2: gradient boosting, depth 3, rate 0.05			
boosting	96 questions	0.4424 ± 0.0049	0.452
boosting	260 columns (final)	0.4440 ± 0.0043	0.454
team blend	trees + networks	0.4464 ± 0.0040*	0.456

Table I
F1 of the positive class. CV F1: mean ± standard deviation over the 5 test folds (seed 42). AIcrowd: test-set F1 of the models we submitted. Setting: hyperparameters, or input of the boosting; L: largest eigenvalue of $X^\top X/N$. *Blend's own protocol (Section IV), not comparable.

The HAREHAB1 answer: The question HAREHAB1 (rehabilitation after a heart attack) is asked only of people who reported a heart attack, in four states. It is answered in 773 training rows, all positive, and in 238 test rows. It is in the provided data, so we use it (as an "answered" flag for the linear models, a bin for the trees), but a real screening could not: without it the logistic regression loses 0.87 percentage points (pp) of F1 (Table II).

Validation and threshold: We use stratified 5-fold cross-validation (fold seed 42). In each fold everything is fitted on the four training folds, including the decision threshold that maximizes F1, and the F1 is measured on the fifth fold. To compare two models we subtract their F1 fold by fold, in pp (1 pp $= 0.01$ of F1). Before tuning the boosting we fixed a rule: a change is kept only if it gains at least 0.30 pp and is positive on at least 4 of 5 folds, both on these folds and on a second split into 5 folds (seed 43).

III. Part 1: the Six Course Methods

The six functions of `implementations.py` follow the course conventions: the squared loss has a factor $1/2$, the logistic losses use labels in $\{0, 1\}$, and the regularized losses are returned without their penalty. The regularized logistic regression minimizes the summed log-loss plus $(\lambda_C/2)\|w\|^2$ with $\lambda_C = 10$, that is $\lambda = \lambda_C/(2N) \approx 1.9 \cdot 10^{-5}$ in the

Change	seed 42	seed 43
Reference: regularized logistic regression		
raw answer codes (no decoding)	-1.28 ± 0.16	
without the HAREHAB1 answer	-0.87 ± 0.12	
without the 43 secondary questions	-0.48 ± 0.21	
without the state	-0.17 ± 0.20	
without missing indicators	-0.02 ± 0.11	
$\lambda_C = 1$ instead of 10	-0.01 ± 0.03	
$\lambda_C = 100$ instead of 10	$+0.03 \pm 0.09$	
boosting on the same 96 questions	$+0.40 \pm 0.30$	$+0.33$
Reference: boosting on the 96 questions		
260 codebook columns (final model)	$+0.16 \pm 0.31$	$+0.35$
average with a neural network	$+0.19 \pm 0.22$	$+0.07$
bagging of 3 models	$+0.05 \pm 0.18$	
64 bins instead of 32	-0.14 ± 0.12	
depth 4 instead of 3	-0.02 ± 0.20	
rate 0.025 instead of 0.05	-0.15 ± 0.26	
trees started from the logistic logits	-0.15 ± 0.18	
Reference: final model		
threshold 0.5 instead of the tuned one	-19.0 ± 0.7	

Table II
PAIRED DIFFERENCE IN F1 (PP): CHANGED MODEL MINUS THE REFERENCE OF ITS BLOCK, ON THE SAME TEST FOLD. SEED 42: MEAN $\pm$ STD OVER THE FOLDS OF TABLE I. SEED 43: MEAN OVER A SECOND SPLIT INTO 5 FOLDS, RUN ONLY FOR THE CANDIDATE CHANGES.

convention of the course ($N \approx 2.6 \cdot 10^5$ training rows). The threshold of these models maximizes F1 on their training predictions.

Table I shows that the loss matters more than the optimizer: the four squared-loss methods reach 0.421–0.424, the two logistic regressions 0.438. The penalty barely matters (λ_C from 1 to 100 moves F1 by at most 0.03 pp), so ridge keeps a small λ and equals least squares. GD (step $0.9/L$, below the divergence limit $2/L$) and SGD, stopped after a fixed number of steps, stay slightly below. Gradient descent on the logistic loss matches Newton's method within 0.01 pp. In the upper block of Table II, decoding the answers, the HAREHAB1 answer and the secondary questions all help; the state and the missing indicators stay within the noise of the folds.

IV. PART 2: GRADIENT BOOSTING

Model: Gradient boosting [2] adds trees that each fit the gradient $g = p - y$ and the curvature $h = p(1-p)$ of the log-loss at the current predictions p. Each column is cut into at most 32 quantile bins plus a bin for missing values. Trees have depth 3; each split also chooses the side of the missing values and keeps a sum of h of at least 5 per side; a leaf predicts $-\sum g/(\sum h + 10)$, an L_2 penalty on the leaf values. Each tree sees a random half of the rows and of the columns and is shrunk by a rate of 0.05. The trees are fitted on 80% of the training rows; the other 20% stop the boosting when their log-loss has not improved for 30 trees (at most 1000) and set the threshold. On the full training set this gives 896 trees and a threshold of probability 0.20.

Results: On the same 96 questions, boosting adds 0.40 pp to the logistic regression, positive on all 5 folds

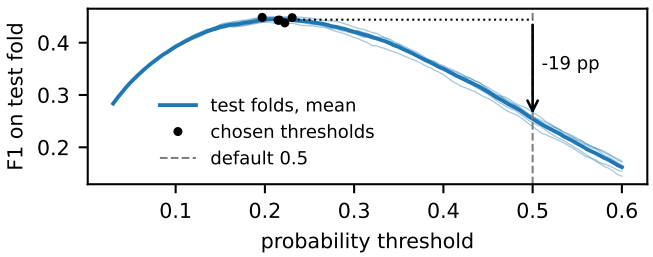


Figure 1. Final model: test F1 of each fold (thin) and their mean (thick) against the probability threshold. Dots: F1 at the threshold chosen on the validation rows. Arrow: mean drop in F1 with the default 0.5.

(Table II). The 260 codebook columns add 0.16 pp on seed 42 (2 of 5 folds positive) and 0.35 pp on seed 43 (5 of 5), so they fail our rule on the first split. We kept them as an exception, decided after also seeing the AIcrowd score: over all 10 folds the gain is +0.26 pp (positive on 7), twice that of averaging with a neural network (64 ReLU units, +0.13 pp), and only the input changes, not the model. The other variants of the lower block did not pass the rule. Fig. 1 shows the largest single lever: the default threshold 0.5 flags too few people and costs 19 pp, while the threshold chosen on the validation rows ($p \approx 0.2$, about 13% flagged) is close to the best one of every fold.

Team blend: Our team's best AIcrowd score, 0.456, comes from a heavier model: boosted trees of depth 6 (300 trees, 4 seeds, weight 0.7) averaged with neural networks with two hidden layers (10 seeds, weight 0.3). Its cross-validated F1, 0.446, uses other folds and a single threshold chosen on the same out-of-fold predictions it is scored on, so it is slightly optimistic. It gains 0.2 pp on AIcrowd but runs for about an hour, so we submitted the simpler model.

V. DISCUSSION

Once the data are decoded, the logistic regression and the boosting differ by less than 0.7 pp, and the gains of the boosting and of the codebook columns are of the order of the fold-to-fold noise (standard error about 0.2 pp [3]). The leaderboard moved with the cross-validation (0.447, 0.452, 0.454) but sits about 0.9 pp higher, and with about 9,700 expected positives its own standard error is about 0.4 pp. Precision stays low (0.38 in cross-validation, at a recall of 0.54): many sick people give the same self-reported answers as healthy ones, and part of what the model "predicts" is a heart attack that people already reported, through HAREHAB1.

Reproducibility: `run.py` reproduces our submitted file (8.5 minutes once the data are cached), `experiments.py` the cross-validation results (except five rejected variants, see the README) and Fig. 1, and `blend/run_blend.py` the blend (about one hour).

REFERENCES

[1] Centers for Disease Control and Prevention, "Behavioral risk factor surveillance system: 2015 codebook report," 2016, https://www.cdc.gov/brfss/annual_data/annual_2015.html.

[2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[3] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.